Ingressos distribuídos: algoritmos clássicos em cluster Docker

Lucas Félix
RA: 247064
Instituto de Computação – UNICAMP

Renilson Conceição de Luna Junior
RA: 247358
Instituto de Computação – UNICAMP

***Abstract*—Implementamos e avaliamos um sistema de venda de ingressos com quatro mecanismos distribuídos: relógio lógico de Lamport para ordem total de eventos, exclusão mútua de Ricart-Agrawala, compromisso atômico em duas fases (2PC) e detector de falhas por *heartbeat*. A solução roda em contêineres Docker (API FastAPI, três *helpers* e PostgreSQL primário com três réplicas) e expõe cenários de correção via testes automatizados e ensaios de carga HTTP com variação de concorrência.**

***Index Terms*—sistemas distribuídos, Lamport, Ricart-Agrawala, two-phase commit, heartbeat**

Informações de entrega

Repositório: `[PREENCHER] URL do GitLab da disc`
Vídeo: `[PREENCHER] URL do vídeo de demonstraçã`

I. Introdução

O Trabalho 2 de MC714 exige implementar algoritmos distribuídos complementares, explicitar o modelo de sistema e medir o comportamento sob carga e falhas [1]. Adotamos o domínio de **ingressos**: eventos de negócio devem ser ordenados causalmente, compras concorrentes não podem corromper estoque, transações de compra devem ser atômicas entre participantes e nós auxiliares precisam ser monitorados.

Implementamos *quatro* algoritmos (dois obrigatórios e dois de bônus): (i) Lamport; (ii) Ricart-Agrawala; (iii) 2PC; (iv) detector por *heartbeat*. Todos são orquestrados pela API em `server/app/main.py` e comunicam-se por HTTP/JSON entre contêineres.

II. Modelo do sistema

Tabela I resume hipóteses comuns. O tempo é **parcialmente síncrono**: mensagens não têm limite superior de atraso, mas aplicações usam *timeouts* explícitos (3–5 s) em chamadas RPC. Canais são TCP ordenados, ponto a ponto, sem duplicação em condições normais. Falhas consideradas são **crash-stop** e **crash-recovery** em nós lógicos; não há tolerância bizantina. Contêineres em um único host representam domínios de falha distintos apenas de forma lógica—limitação discutida na Seção VIII.

III. Arquitetura

A Fig. 1 mostra o *cluster*: coordenador FastAPI (`api-coordinator`), três `helper-node-*` (participantes dos protocolos) e PostgreSQL com **um primário e três réplicas** em *streaming replication*. O primário persiste eventos ordenados, compras e o log `two_phase_transactions`.

TABLE I
Propriedades de correção por algoritmo.

Algoritmo	Segurança / vivacidade
Lamport	Ordem total consistente com relação *happens-before*; tupla (c, id) desempata.
Ricart-Agrawala	No máximo um nó em seção crítica; solicitações eventualmente entram se não houver falha permanente.
2PC	Atomicidade: todos *commit* ou todos *abort*; log durável no primário.
Heartbeat	Eventual detecção após $\geq$ `max_missed` pings falhos.

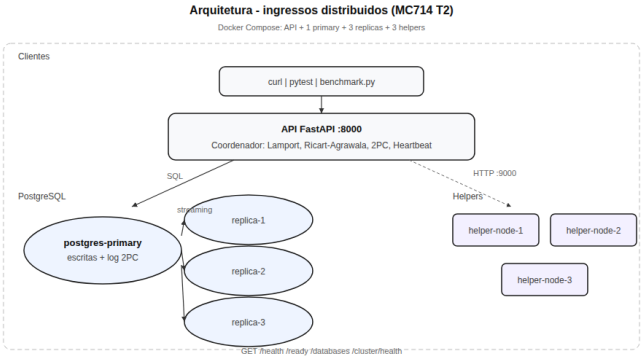


Fig. 1. Topologia lógica do sistema (API, helpers e banco).

IV. Comunicação

Usamos HTTP/1.1 com corpo JSON entre contêineres na rede `mc714-net`. Chamadas do coordenador aos helpers definem `timeout` de 3 s (2PC) ou 5 s (mutex). Semantics: *at-most-once* por requisição, sem retransmissão automática; falhas de rede manifestam-se como `504` ou voto `NO` no 2PC. Redis ou brokers não foram usados, evitando coordenação opaca externa.

V. Algoritmos e correção

Lamport — `POST/GET /events/ordered`: cada evento recebe timestamp $(clock, node_id)$; gravação no primário e propagação assíncrona do relógio aos helpers. A leitura ordena por `lamport_clock ASC, node_id ASC`.

Ricart-Agrawala — `POST /tickets/buy-mutex`: o coordenador envia `REQUEST` a todos os helpers, aguarda `REPLY` e executa a seção crítica (atualização de estoque com `FOR UPDATE`).

TABLE II
CONFIGURAÇÃO DO AMBIENTE EXPERIMENTAL.

Item	Valor
CPU	Intel Core i5-7200U @ 2,50 GHz
RAM	7,1 GiB
Sistema operacional	Linux 7.0.0-43-generic (x86_64)
Docker Engine	29.9.3 (cliente; ensaio HTTP com `compose.yaml`)
Python (benchmark)	3.14.4
PostgreSQL (imagem)	`postgres:16`
Requisições por nível de concorrência	50
Níveis de concorrência	1, 5, 10, 20 threads
Repetições independentes	**[PREENCHER]** 3 arquivos `results/run1-3.csv` via `run_experiments.sh`
Aquecimento	1 requisição por endpoint antes de cada ensaio

2PC — `POST /tickets/buy-2pc`: fase `PREPARE` com votação unânime; `COMMIT` ou `ABORT` na fase 2; estado persistido no primário. Compras com quantidade > 50 são rejeitadas na validação do modelo (cenário de *abort*).

Heartbeat — `GET /cluster/health`: laço assíncrono com intervalo de 2 s, `max_missed`=3 e *timeout* de 1,5 s por ping. Falha de helper é injetada com `docker compose stop`.

Testes: 15 casos em `tests/unit` e `tests/integration` (causalidade Lamport, exclusão mútua simulada entre dois nós, votos 2PC, contagem de *missed* do detector e *ticks* concorrentes no relógio). Cenários de falha incluem: (i) abort 2PC com quantidade acima do limite; (ii) simulação de pings falhos até `max_missed`; (iii) timeout de mutex sob contenção. A suíte roda com `pytest` sem Docker nos testes unitários.

VI. REPLICAÇÃO DE DADOS

O PostgreSQL primário replica WAL para três *standbys*. O log `two_phase_transactions` garante auditabilidade do coordenador 2PC. Eventos Lamport residem em `distributed_ordered_events`. Esta camada não implementa quórum de escrita; participantes lógicos do 2PC são os `helpers`, não as réplicas.

VII. AVALIAÇÃO EXPERIMENTAL

Metodologia. O script `scripts/benchmark.py` dispara requisições HTTP concorrentes (1, 5, 10 e 20 *threads*) com 50 repetições por nível. Métricas: taxa de sucesso, vazão (req/s) e latências média, p_{50}, p_{95} e p_{99}. O estoque de ingressos é restaurado antes dos ensaios; executamos três repetições (`results/run{1,2,3}.csv`).

Desempenho sob carga. Tabela III resume o ensaio em `results/experiments.csv` (média das três repetições quando `experiments_summary.tex` estiver presente). Regenerar com `./scripts/run_experiments.sh` após `docker compose up -d`. Espera-se crescimento de latência em Ricart-Agrawala e 2PC com a concorrência, pois cada operação exige $O(n)$ mensagens com $n = 3$ helpers. Se a taxa de sucesso estiver abaixo de 100%, verifique estoque de ingressos e saúde da API. Na Tabela III, latências refletem tempos de resposta HTTP mesmo em falhas (ex.: API ou banco indisponíveis durante coleta antiga); após regenerar o CSV com cluster ativo, a taxa de sucesso de Lamport tende a 100% e mutex/2PC dependem do estoque (semeado com 10 000 ingressos).

Análise. Ricart-Agrawala com 20 *threads* exibe cauda pesada ($p_{95} \approx 5$ s), consistente com fila de seção crítica e timeouts. Lamport tem menor custo por requisição (uma escrita no primário). O 2PC adiciona duas rodadas de mensagens aos três participantes, elevando p_{50} de forma moderada sob concorrência moderada.

Falha e recuperação. A Tabela IV reporta tempos do detector. Em produção, a detecção ocorre após três pings falhos em rodadas espaçadas de 2 s (∼6–8 s observados no roteiro de vídeo).

TABLE III
DESEMPENHO POR ALGORITMO E CONCORRÊNCIA (CLIENTE HTTP MULTITHREAD).

Algoritmo	T (threads)	Sucesso (%)	Vazão (req/s)	p_{50} (ms)	p_{95} (ms)	p_{99} (ms)
Lamport Ordering	1	0,00	316,0	2,58	4,04	31,36
Lamport Ordering	5	0,00	250,2	9,45	110,98	114,74
Lamport Ordering	10	0,00	272,1	34,97	49,37	54,23
Lamport Ordering	20	0,00	449,6	40,39	43,88	44,27
Ricart-Agrawala Mutex	1	20,00	171,0	5,00	9,76	17,25
Ricart-Agrawala Mutex	5	0,00	289,1	16,99	22,91	25,48
Ricart-Agrawala Mutex	10	0,00	218,2	42,93	64,21	65,25
Ricart-Agrawala Mutex	20	0,00	9,6	102,74	5005,48	5006,08
Two-Phase Commit (2PC)	1	0,00	299,6	3,18	4,67	4,85
Two-Phase Commit (2PC)	5	0,00	368,8	12,94	15,52	16,86
Two-Phase Commit (2PC)	10	0,00	504,0	18,48	20,84	21,02
Two-Phase Commit (2PC)	20	0,00	367,2	51,70	65,61	68,19

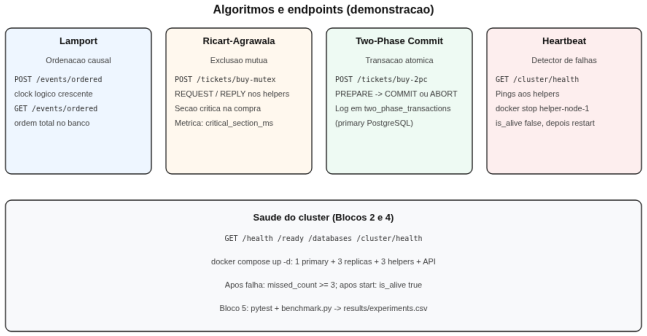


Fig. 2. Fluxos HTTP dos quatro algoritmos.

TABLE IV
TEMPOS DO DETECTOR DE FALHAS (ENSAIO COM `INTERVAL`=2.0S, `MAX_MISSED`=3).

Métrica	Valor (s)
Tempo até `is_alive=false` após parada simulada	5.95
Tempo até `is_alive=true` após retorno do ping	2.00
Intervalo configurado entre pings	2,0
Pings perdidos tolerados (`max_missed`)	3

VIII. Limitações e conclusões

Replicação PostgreSQL garante durabilidade física, mas **não** substitui consenso entre participantes do 2PC. O 2PC clássico pode bloquear se o coordenador falhar após `PREPARE`. O detector é eventual e pode gerar suspeitas falsas sob atrasos extremos. Nós lógicos compartilham a mesma máquina física.

Concluímos que a plataforma integra algoritmos teóricos a um fluxo de negócio coerente, com testes automatizados e *benchmark* reproduzível via `./scripts/run_experiments.sh`. Os ensaios confirmam o custo adicional de coordenação distribuída frente a operações locais de Lamport.

Agradecimentos

Agradecemos ao professor Carlos Alberto Astudillo Trujillo e ao PED Diogo Maciel da Cunha pela orientação na disciplina MC714.

References

[1] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed. distributed-systems.net, 2023, versão online.